

Perceptron Trick

1 Intro

The **perceptron algorithm** is not just about making predictions — it must **learn** from mistakes.

This learning process is called the **Perceptron Trick** (weight update rule).

2 Perceptron Trick

Idea: If the perceptron predicts wrong, **move the decision boundary** (line/plane) slightly so next time, it gets closer to the correct answer.

The update rule:

$$w_{new} = w_{old} + \Delta w$$

Where:

$$\Delta w = \eta \times (y - \hat{y}) \times x$$

- w = weights
- η = learning rate (small positive number)
- y = actual label
- $\hat{y}$ = predicted output
- x = input features

If prediction = correct $\rightarrow$ no change.

If prediction = wrong $\rightarrow$ nudge weights toward correct answer.

3 How to Label Regions?

- The perceptron separates data into **two regions** with a line (in 2D).
- One side of the line = class 1, other side = class 0.
- Labels help the perceptron know **which side a point should belong to**.

For example:

- Red points = 1
- Blue points = 0

The algorithm shifts the line until most points fall in the right region.

4 Transformations

Sometimes, data is not linearly separable (like XOR).

- One trick is to **transform inputs** into higher dimensions, where a line (or plane) *can* separate them.
- Example: Add new features (like $x_1 \times x_2$) to make separation possible.

This idea inspired **kernel methods** and later deep networks.

5 Coding the Algorithm

Simplified pseudocode:

<i># Initialize weights and bias</i>
<i>weights = random values</i>
<i>bias = random value</i>
<i>learning_rate = 0.1</i>
<i># Repeat for multiple epochs</i>
<i>for epoch in range(num_epochs):</i>
<i>for each (x, y) in training_data:</i>
<i># Forward pass</i>
<i>z = np.dot(weights, x) + bias</i>
<i>y_pred = 1 if z > 0 else 0</i>
<i># Error</i>
<i>error = y - y_pred</i>
<i># Update weights</i>
<i>weights = weights + learning_rate * error * x</i>
<i>bias = bias + learning_rate * error</i>

6 Simplified Algorithm

The perceptron can be summarized as:

1. Guess (prediction).
2. Check (compare with actual label).
3. Update (adjust weights if wrong).
4. Repeat (until line fits data well).

This is **early gradient descent** in action.

7 Code Demo

In practice:

- Start with random line.
 - After several iterations, the line rotates/shifts until it correctly separates the data points.
 - Visual demos usually show the boundary moving step by step.
-

8 Outro

- Perceptron learning = **trial and error** → keep adjusting weights until it gets better.
 - Limitations:
 1. Works only if data is linearly separable.
 2. Can't solve XOR or complex problems (needs multiple layers).
 - But it set the foundation for **gradient descent** and **modern deep learning**.
-

Quick Recap

- **Perceptron Trick** = weight update when wrong.
 - **Weight Update Rule:** $\Delta w = \eta(y - y^*)x$
 - **Training** = forward pass → error → update → repeat.
 - Foundation for **all neural network training today**.
-